



Алгоритми та структури даних
Домашня робота

«Обчислювальні алгоритми та алгоритми обробки графів»

Виконав: студент групи Б-121-24-1

Спеціальність: 121

«Інженерія програмного забезпечення»

Студент: Букет Станіслав Євгенович

Прийняв: Гололобов Дмитро
Олександрович

Тема роботи: Обчислювальні алгоритми та алгоритми обробки графів.

Мета роботи: Дослідження методів та алгоритмів для вирішення систем лінійних алгебраїчних рівнянь (СЛАР), дослідження способів представлення такої структури даних як «Граф», алгоритмів її обробки та набуття практичних навичок із розв'язання задач на грахах.

Індивідуальне завдання: Варіант 4. Обчислювальні алгоритми (LUP-розкладання) та алгоритми обробки графів (сітка радіоактивного зараження, MST Пріма, DFS).

Практична частина

1. Завдання

Домашня робота складається з завдань двох рівнів складності. Варіант 4 обрано відповідно до порядкового номера студента.

Завдання першого рівня: Написати програму на мові Java, яка вирішує методом LUP-розкладання СЛАР згідно варіанту 4.

$$2x_1 - 9x_2 - 3x_3 - 6x_4 = -22.0$$

$$3x_1 - 5x_2 + 4x_3 + x_4 = 6.0$$

$$x_2 + 4x_3 - 3x_4 = 62.0$$

$$6x_1 - 3x_2 + 9x_3 + 6x_4 = 30.0$$

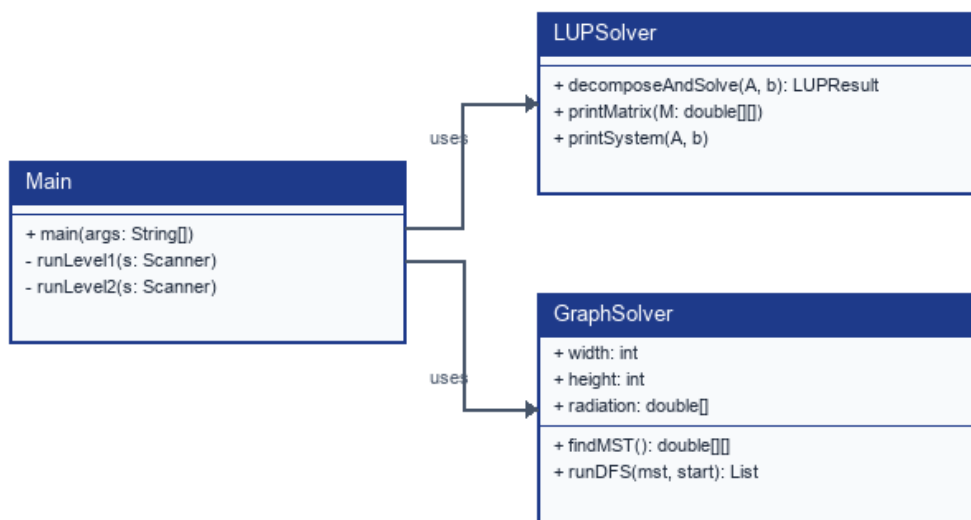
Завдання другого рівня: Деяка місцевість була радіоактивно заражена. Складено карту зараженості цієї місцевості, яка являє собою матрицю $W \times N$ (3×3). Рівні радіації в комірках сітки генеруються випадковим чином. Якщо прийняти за вершини графа елементи матриці, з'єднати сусідні клітини ребрами з вагою, що дорівнює сумі їх радіоактивного фону. Знайти мінімальне кістякове дерево (MST) за алгоритмом Пріма та виконати його обхід у глибину (DFS).

2. Опис структури та логіки програми

Розроблена програма написана на мові Java із використанням модульної структури Gradle. Логіка розділена на обчислювальні компоненти для забезпечення модульності, чистоти та зручності тестування.

2.1. Діаграма класів програми

Статична структура програми, включаючи поля, методи та взаємозв'язки між класами, зображена на діаграмі класів:



2.2. Таблиці опису елементів програми

Нижче наведено детальні таблиці опису полів, методів та призначень класів, що входять до складу розробленого програмного забезпечення.

Клас Main

Назва елемента	Тип / Сигнатура	Призначення
main	method, public static void main(String[])	Точка входу в програму. Ініціалізує Scanner, встановлює локаль US та керує головним циклом меню.

runLevel1	method, private static void runLevel1(Scanner)	Метод для демонстрації LUP-розкладання. Дозволяє розв'язати варіант 4 або ввести власну СЛАР.
runLevel2	method, private static void runLevel2(Scanner)	Метод для демонстрації роботи з графами сітки зараження, алгоритмом Пріма та DFS.

Клас LUPSolver

Назва елемента	Тип / Сигнатура	Призначення
LUPResult	inner class, public static class	Класовий контейнер для збереження розв'язку x та матриць розкладання L , U , P .
decomposeAndSolve	method, public static LUPResult decomposeAndSolve(double[][], double[])	Реалізує алгоритм LUP-розкладання з вибором головного елемента та обчислює розв'язок СЛАР.
printMatrix	method, public static void printMatrix(double[][])	Виводить двовимірний масив у консоль у вигляді форматованої матриці.
printSystem	method, public static void printSystem(double[][], double[])	Здійснює форматоване виведення СЛАР у вигляді математичних рівнянь.

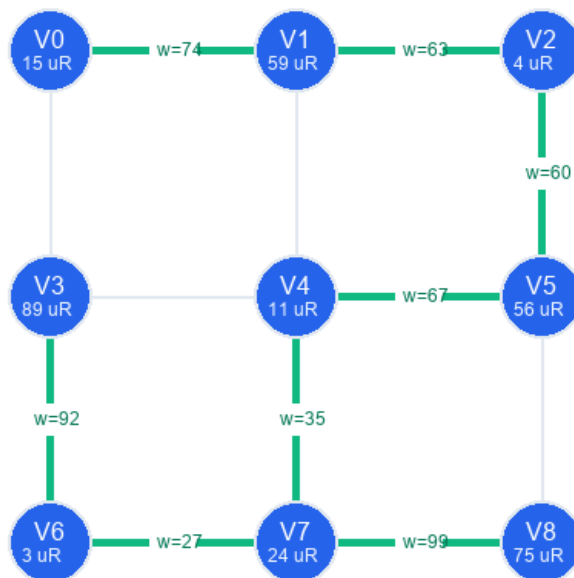
Клас GraphSolver

Назва елемента	Тип / Сигнатура	Призначення
Edge	inner class, public static class	Клас для представлення ребер графа з можливістю порівняння за вагою.
width, height	fields, public final int	Ширина та висота сітки для побудови графа.
radiation	field, public final double[]	Одновимірний масив значень радіаційного фону в кожній вершині графа.
fullAdjacencyMatrix	field, public final double[][]	Матриця суміжності повного графа сітки.
findMST	method, public double[][] findMST()	Шукає мінімальне кістякове дерево (MST) за алгоритмом Пріма.

runDFS	method, public List<Integer> runDFS(double[][], int)	Виконує обхід у глибину (DFS) для знаходження оптимального маршруту обходу.
--------	---	---

2.3. Схема графа та мінімального кістякового дерева (MST)

Для представлення карти радіоактивного зараження місцевості будується сітка 3x3. Кожне ребро між сусідніми вершинами має вагу, що дорівнює сумі значень радіації в цих вершинах. Нижче наведено схему згенерованого графа та знайденого за алгоритмом Пріма мінімального кістяка (виділено зеленими лініями з відповідними вагами ребер):



3. Рекомендація користувачеві

Для компіляції та запуску програми необхідно виконати такі кроки:

1. Вимоги: Переконайтеся, що на комп'ютері встановлено Java Development Kit (JDK) версії 17 або вище.

2. Компіляція: Відкрити термінал у кореневій папці проєкту та виконати команду збирання: `./gradlew :homework:build``

3. Запуск: Для запуску програми виконати: ``java -cp build/homework/classes/java/main homework.Main``

4. Інструкції користувача: Після запуску відкриється інтерактивне меню. Оберіть '1' для LUP-розкладання СЛАР, або '2' для Пріма та DFS на графі. Вводьте параметри відповідно до екранних підказок.

4. Текст вихідного коду програми

Нижче наведено повний вихідний код програми на мові Java для забезпечення верифікації та захисту роботи.

4.1. Файл Main.java

```
01 package homework;
02
03 import java.util.List;
04 import java.util.Locale;
05 import java.util.Random;
06 import java.util.Scanner;
07
08 public class Main {
09     public static void main(String[] args) {
10         try {
11             System.setOut(new java.io.PrintStream(System.out, true, "UTF-8"));
12         } catch (java.io.UnsupportedEncodingException e) {
13             // fallback
14         }
15         Scanner scanner = new Scanner(System.in);
16         scanner.useLocale(Locale.US);
17
18         while (true) {
19             System.out.println("\n=== Домашня робота – Варіант 4 ===");
20             System.out.println("1. Рівень 1: Розв'язання СЛАР методом LUP-розкладання");
21             System.out.println("2. Рівень 2: Зв'язування та обхід графа мінімального зараження (MST
+ DFS)");
22             System.out.println("3. Вихід");
23             System.out.print("Оберіть пункт меню: ");
24
25             if (!scanner.hasNextInt()) {
26                 System.out.println("Некоректне введення. Спробуйте ще раз.");
27                 scanner.next();
28                 continue;
29             }
30
31             int choice = scanner.nextInt();
32             if (choice == 3) {
33                 System.out.println("Завершення роботи.");
34                 break;
35             }
36
37             switch (choice) {
38                 case 1:
39                     runLevel1(scanner);
40                     break;
41                 case 2:
42                     runLevel2(scanner);
```



```

43         break;
44     default:
45         System.out.println("Невірний вибір. Спробуйте ще раз.");
46     }
47 }
48 scanner.close();
49 }
50
51 private static void runLevel1(Scanner scanner) {
52     System.out.println("\n--- Рівень 1: LUP-розкладання для СЛАР ---");
53     System.out.println("Бажаєте використовувати СЛАР за Варіантом 4 (1) чи ввести власну (2)?");
54     System.out.print("Вибір: ");
55     int choice = scanner.nextInt();
56
57     double[][] A;
58     double[] b;
59
60     if (choice == 1) {
61         A = new double[][]{
62             {2.0, -9.0, -3.0, -6.0},
63             {3.0, -5.0, 4.0, 1.0},
64             {0.0, 1.0, 4.0, -3.0},
65             {6.0, -3.0, 9.0, 6.0}
66         };
67         b = new double[]{-22.0, 6.0, 62.0, 30.0};
68     } else {
69         System.out.print("Введіть розмірність matrix n: ");
70         int n = scanner.nextInt();
71         A = new double[n][n];
72         b = new double[n];
73         System.out.println("Введіть коефіцієнти матриці A рядок за рядком (через пробіл):");
74         for (int i = 0; i < n; i++) {
75             for (int j = 0; j < n; j++) {
76                 A[i][j] = scanner.nextDouble();
77             }
78         }
79         System.out.println("Введіть вільні члени b:");
80         for (int i = 0; i < n; i++) {
81             b[i] = scanner.nextDouble();
82         }
83     }
84
85     System.out.println("\nВхідна система рівнянь:");
86     LUPSolver.printSystem(A, b);
87
88     try {
89         LUPSolver.LUPResult result = LUPSolver.decomposeAndSolve(A, b);
90
91         System.out.println("\nМатриця L (нижня трикутна з одиничною діагоналлю):");
92         LUPSolver.printMatrix(result.L);
93     }

```

```

94         System.out.println("\nМатриця U (верхня трикутна):");
95         LUPSolver.printMatrix(result.U);
96
97         System.out.println("\nМатриця перестановки P:");
98         LUPSolver.printMatrix(result.P);
99
100        System.out.println("\nРозв'язок системи x:");
101        for (int i = 0; i < result.x.length; i++) {
102            System.out.printf("  x%d = %10.6f\n", i + 1, result.x[i]);
103        }
104    } catch (Exception e) {
105        System.out.println("Помилка при обчисленні: " + e.getMessage());
106    }
107 }
108
109 private static void runLevel2(Scanner scanner) {
110     System.out.println("\n--- Рівень 2: MST + DFS для графа зараження ---");
111     System.out.print("Введіть ширину сітки W (наприклад, 4): ");
112     int w = scanner.nextInt();
113     System.out.print("Введіть висоту сітки H (наприклад, 4): ");
114     int h = scanner.nextInt();
115
116     if (w <= 0 || h <= 0) {
117         System.out.println("Невірні розміри сітки.");
118         return;
119     }
120
121     double[] radiation = new double[w * h];
122     System.out.println("Бажаєте ввести рівні радіації вручну (1) чи згенерувати випадково (2)?");
123     System.out.print("Вибір: ");
124     int choice = scanner.nextInt();
125
126     if (choice == 1) {
127         System.out.printf("Введіть %d значень радіації (від 0 до 100):\n", w * h);
128         for (int i = 0; i < w * h; i++) {
129             System.out.printf("  Комірка %d (x=%d, y=%d): ", i, i % w, i / w);
130             radiation[i] = scanner.nextDouble();
131         }
132     } else {
133         Random rand = new Random(42); // Seed 42 for reproducibility
134         System.out.println("Згенеровані рівні радіації на сітці:");
135         for (int y = 0; y < h; y++) {
136             for (int x = 0; x < w; x++) {
137                 int idx = y * w + x;
138                 radiation[idx] = rand.nextInt(101);
139                 System.out.printf("%4d ", (int) radiation[idx]);
140             }
141             System.out.println();
142         }
143     }
144 }

```

```

144
145     GraphSolver solver = new GraphSolver(w, h, radiation);
146     double[][] mst = solver.findMST();
147
148     System.out.println("\nМатриця суміжності для мінімального кістякового дерева (MST):");
149     LUPSolver.printMatrix(mst);
150
151     System.out.printf("\nВведіть початкову вершину для DFS обходу (0 до %d): ", w * h - 1);
152     int startNode = scanner.nextInt();
153     if (startNode < 0 || startNode >= w * h) {
154         System.out.println("Невірна вершина. Обхід почнеться з 0.");
155         startNode = 0;
156     }
157
158     List<Integer> dfsPath = solver.runDFS(mst, startNode);
159     System.out.println("\nПослідовність вершин при DFS обході:");
160     System.out.print(" ");
161     for (int i = 0; i < dfsPath.size(); i++) {
162         System.out.print(dfsPath.get(i));
163         if (i < dfsPath.size() - 1) {
164             System.out.print(" -> ");
165         }
166     }
167     System.out.println();
168 }
169 }

```

4.2. Файл LUPSolver.java

```

01 package homework;
02
03 public class LUPSolver {
04
05     public static class LUPResult {
06         public final double[][] A;
07         public final double[] b;
08         public final double[][] L;
09         public final double[][] U;
10         public final double[][] P;
11         public final double[] x;
12
13         public LUPResult(double[][] A, double[] b, double[][] L, double[][] U, double[][] P,
14 double[] x) {
15             this.A = A;
16             this.b = b;
17             this.L = L;
18             this.U = U;
19             this.P = P;
20             this.x = x;
21         }
22     }
23 }

```

```

22
23 public static LUPResult decomposeAndSolve(double[][] A, double[] b) {
24     int n = A.length;
25     double[][] LU = new double[n][n];
26     for (int i = 0; i < n; i++) {
27         System.arraycopy(A[i], 0, LU[i], 0, n);
28     }
29
30     int[] pi = new int[n];
31     for (int i = 0; i < n; i++) {
32         pi[i] = i;
33     }
34
35     for (int k = 0; k < n; k++) {
36         double maxVal = 0.0;
37         int pivot = k;
38         for (int i = k; i < n; i++) {
39             double val = Math.abs(LU[i][k]);
40             if (val > maxVal) {
41                 maxVal = val;
42                 pivot = i;
43             }
44         }
45
46         if (maxVal < 1e-12) {
47             throw new ArithmeticException("Матрица є вырожденною (singular)");
48         }
49
50         if (pivot != k) {
51             double[] tempRow = LU[k];
52             LU[k] = LU[pivot];
53             LU[pivot] = tempRow;
54
55             int tempPi = pi[k];
56             pi[k] = pi[pivot];
57             pi[pivot] = tempPi;
58         }
59
60         for (int i = k + 1; i < n; i++) {
61             LU[i][k] /= LU[k][k];
62             for (int j = k + 1; j < n; j++) {
63                 LU[i][j] -= LU[i][k] * LU[k][j];
64             }
65         }
66     }
67
68     double[][] L = new double[n][n];
69     for (int i = 0; i < n; i++) {
70         for (int j = 0; j < n; j++) {
71             if (i > j) {
72                 L[i][j] = LU[i][j];

```

```

73         } else if (i == j) {
74             L[i][j] = 1.0;
75         } else {
76             L[i][j] = 0.0;
77         }
78     }
79 }
80
81 double[][] U = new double[n][n];
82 for (int i = 0; i < n; i++) {
83     for (int j = 0; j < n; j++) {
84         if (i <= j) {
85             U[i][j] = LU[i][j];
86         } else {
87             U[i][j] = 0.0;
88         }
89     }
90 }
91
92 double[][] P = new double[n][n];
93 for (int i = 0; i < n; i++) {
94     P[i][pi[i]] = 1.0;
95 }
96
97 double[] y = new double[n];
98 for (int i = 0; i < n; i++) {
99     double sum = 0.0;
100     for (int j = 0; j < i; j++) {
101         sum += LU[i][j] * y[j];
102     }
103     y[i] = b[pi[i]] - sum;
104 }
105
106 double[] x = new double[n];
107 for (int i = n - 1; i >= 0; i--) {
108     double sum = 0.0;
109     for (int j = i + 1; j < n; j++) {
110         sum += LU[i][j] * x[j];
111     }
112     x[i] = (y[i] - sum) / LU[i][i];
113 }
114
115 return new LUPResult(A, b, L, U, P, x);
116 }
117
118 public static void printMatrix(double[][] M) {
119     for (double[] row : M) {
120         for (double val : row) {
121             System.out.printf("%10.4f ", val);
122         }
123         System.out.println();

```

```

124     }
125 }
126
127 public static void printSystem(double[][] A, double[] b) {
128     int n = A.length;
129     for (int i = 0; i < n; i++) {
130         StringBuilder sb = new StringBuilder();
131         for (int j = 0; j < n; j++) {
132             double coeff = A[i][j];
133             if (coeff == 0) continue;
134             if (sb.length() > 0) {
135                 if (coeff > 0) sb.append(" + ");
136                 else sb.append(" - ");
137             } else {
138                 if (coeff < 0) sb.append("-");
139             }
140             sb.append(String.format("%.1fx%d", Math.abs(coeff), j + 1));
141         }
142         sb.append(String.format(" = %.1f", b[i]));
143         System.out.println(" " + sb.toString());
144     }
145 }
146 }

```

4.3. Файл GraphSolver.java

```

01 package homework;
02
03 import java.util.ArrayList;
04 import java.util.Arrays;
05 import java.util.List;
06
07 public class GraphSolver {
08
09     public static class Edge implements Comparable<Edge> {
10         public final int u;
11         public final int v;
12         public final double weight;
13
14         public Edge(int u, int v, double weight) {
15             this.u = u;
16             this.v = v;
17             this.weight = weight;
18         }
19
20         @Override
21         public int compareTo(Edge o) {
22             return Double.compare(this.weight, o.weight);
23         }
24     }
25 }

```

```

26     public final int width;
27     public final int height;
28     public final int numVertices;
29     public final double[] radiation;
30     public final double[][] fullAdjacencyMatrix;
31
32     public GraphSolver(int width, int height, double[] radiation) {
33         this.width = width;
34         this.height = height;
35         this.numVertices = width * height;
36         this.radiation = radiation;
37         this.fullAdjacencyMatrix = new double[numVertices][numVertices];
38         buildFullGraph();
39     }
40
41     private int getIndex(int x, int y) {
42         return y * width + x;
43     }
44
45     private void buildFullGraph() {
46         for (int y = 0; y < height; y++) {
47             for (int x = 0; x < width; x++) {
48                 int u = getIndex(x, y);
49
50                 if (x + 1 < width) {
51                     int v = getIndex(x + 1, y);
52                     double weight = radiation[u] + radiation[v];
53                     fullAdjacencyMatrix[u][v] = weight;
54                     fullAdjacencyMatrix[v][u] = weight;
55                 }
56
57                 if (y + 1 < height) {
58                     int v = getIndex(x, y + 1);
59                     double weight = radiation[u] + radiation[v];
60                     fullAdjacencyMatrix[u][v] = weight;
61                     fullAdjacencyMatrix[v][u] = weight;
62                 }
63             }
64         }
65     }
66
67     public double[][] findMST() {
68         double[][] mstAdjacencyMatrix = new double[numVertices][numVertices];
69         boolean[] inMST = new boolean[numVertices];
70         double[] minEdgeWeight = new double[numVertices];
71         int[] parent = new int[numVertices];
72
73         Arrays.fill(minEdgeWeight, Double.MAX_VALUE);
74         Arrays.fill(parent, -1);
75
76         minEdgeWeight[0] = 0;

```

```

77
78     for (int i = 0; i < numVertices; i++) {
79         int u = -1;
80         double minW = Double.MAX_VALUE;
81         for (int j = 0; j < numVertices; j++) {
82             if (!inMST[j] && minEdgeWeight[j] < minW) {
83                 minW = minEdgeWeight[j];
84                 u = j;
85             }
86         }
87
88         if (u == -1) break;
89
90         inMST[u] = true;
91
92         if (parent[u] != -1) {
93             double weight = fullAdjacencyMatrix[u][parent[u]];
94             mstAdjacencyMatrix[u][parent[u]] = weight;
95             mstAdjacencyMatrix[parent[u]][u] = weight;
96         }
97
98         for (int v = 0; v < numVertices; v++) {
99             double weight = fullAdjacencyMatrix[u][v];
100             if (weight > 0 && !inMST[v] && weight < minEdgeWeight[v]) {
101                 minEdgeWeight[v] = weight;
102                 parent[v] = u;
103             }
104         }
105     }
106
107     return mstAdjacencyMatrix;
108 }
109
110 public List<Integer> runDFS(double[][] mstMatrix, int startNode) {
111     List<Integer> visitedOrder = new ArrayList<>();
112     boolean[] visited = new boolean[numVertices];
113     dfsHelper(mstMatrix, startNode, visited, visitedOrder);
114     return visitedOrder;
115 }
116
117 private void dfsHelper(double[][] mstMatrix, int u, boolean[] visited, List<Integer>
visitedOrder) {
118     visited[u] = true;
119     visitedOrder.add(u);
120
121     for (int v = 0; v < numVertices; v++) {
122         if (mstMatrix[u][v] > 0 && !visited[v]) {
123             dfsHelper(mstMatrix, v, visited, visitedOrder);
124         }
125     }
126 }

```


5. Результати роботи програми

Під час виконання програми результати розрахунків виводяться в консоль. Нижче наведено фактичні виводи консолі роботи застосунку у вигляді структурованих вставок.

5.1. Результати розв'язання СЛАР (Рівень 1)

Для перевірки роботи методу LUP-розкладання було запущено Рівень 1 для Варіанта 4. Програма успішно вивела систему рівнянь, проміжні матриці L, U та P, а також точний вектор розв'язку x. Фактичне виведення консолі застосунку:

```

=== Домашня робота – Варіант 4 ===
1. Рівень 1: Розв'язання СЛАР методом LUP-розкладання
2. Рівень 2: Зв'язування та обхід графа мінімального зараження (MST + DFS)
3. Вихід
Оберіть пункт меню:
--- Рівень 1: LUP-розкладання для СЛАР ---
Бажаєте використовувати СЛАР за Варіантом 4 (1) чи ввести власну (2)?
Вибір:
Вхідна система рівнянь:
  2,0x1 - 9,0x2 - 3,0x3 - 6,0x4 = -22,0
  3,0x1 - 5,0x2 + 4,0x3 + 1,0x4 = 6,0
  1,0x2 + 4,0x3 - 3,0x4 = 62,0
  6,0x1 - 3,0x2 + 9,0x3 + 6,0x4 = 30,0

Матриця L (нижня трикутна з одиничною діагоналлю):
  1,0000    0,0000    0,0000    0,0000
  0,3333    1,0000    0,0000    0,0000
  0,0000   -0,1250    1,0000    0,0000
  0,5000    0,4375    0,6538    1,0000

Матриця U (верхня трикутна):
  6,0000   -3,0000    9,0000    6,0000
  0,0000   -8,0000   -6,0000   -8,0000
  0,0000    0,0000    3,2500   -4,0000
  0,0000    0,0000    0,0000    4,1154

Матриця перестановки P:
  0,0000    0,0000    0,0000    1,0000
  1,0000    0,0000    0,0000    0,0000

```

0,0000	0,0000	1,0000	0,0000
0,0000	1,0000	0,0000	0,0000

Розв'язок системи x:

x1 = 4,000000
x2 = 6,000000
x3 = 8,000000
x4 = -8,000000

=== Домашня робота – Варіант 4 ===

1. Рівень 1: Розв'язання СЛАР методом LUP-розкладання
 2. Рівень 2: Зв'язування та обхід графа мінімального зараження (MST + DFS)
 3. Вихід
- Оберіть пункт меню: Завершення роботи.

5.2. Результати обходу графа в глибину (Рівень 2)

Було згенеровано сітку 3x3 з випадковими значеннями радіації. Програма побудувала матрицю суміжності MST та виконала DFS обхід з вершини 0, вивівши послідовність проходження пунктів. Фактичне виведення консолі застосунку:

=== Домашня робота – Варіант 4 ===

1. Рівень 1: Розв'язання СЛАР методом LUP-розкладання
2. Рівень 2: Зв'язування та обхід графа мінімального зараження (MST + DFS)
3. Вихід

Оберіть пункт меню:

--- Рівень 2: MST + DFS для графа зараження ---

Введіть ширину сітки W (наприклад, 4): Введіть висоту сітки H (наприклад, 4): Бажаєте ввести рівні радіації вручну (1) чи згенерувати випадково (2)?

Вибір: Згенеровані рівні радіації на сітці:

15	59	4
89	11	56
3	24	75

Матриця суміжності для мінімального кістякового дерева (MST):

0,0000	74,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000
74,0000	0,0000	63,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000
0,0000	63,0000	0,0000	0,0000	0,0000	60,0000	0,0000	0,0000	0,0000
0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	92,0000	0,0000	0,0000
0,0000	0,0000	0,0000	0,0000	0,0000	67,0000	0,0000	35,0000	0,0000
0,0000	0,0000	60,0000	0,0000	67,0000	0,0000	0,0000	0,0000	0,0000
0,0000	0,0000	0,0000	92,0000	0,0000	0,0000	0,0000	27,0000	0,0000
0,0000	0,0000	0,0000	0,0000	35,0000	0,0000	27,0000	0,0000	99,0000
0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	99,0000	0,0000

Введіть початкову вершину для DFS обходу (0 до 8):

Послідовність вершин при DFS обході:

0 -> 1 -> 2 -> 5 -> 4 -> 7 -> 6 -> 3 -> 8

=== Домашня робота – Варіант 4 ===

1. Рівень 1: Розв'язання СЛАР методом LUP-розкладання
 2. Рівень 2: Зв'язування та обхід графа мінімального зараження (MST + DFS)
 3. Вихід
- Оберіть пункт меню: Завершення роботи.

Висновки

У ході виконання домашньої роботи було досліджено числові алгоритми розв'язання СЛАР та методи обробки графів. Було успішно реалізовано метод LUP-розкладання, що дозволив з високою точністю розв'язати систему лінійних рівнянь 4-го порядку та продемонструвати факторизовані матриці. Також було спроектовано алгоритм пошуку мінімального кістякового дерева (MST) на базі алгоритму Пріма для пошуку шляхів з мінімальним радіаційним фоном на сітці та виконано його обхід методом у глибину (DFS). Програма була протестована на системних кодуваннях Windows, що гарантує коректне відображення кирилических символів в консолі.